

Package `rpbnb.tmb`

Random-Parameter Bivariate Negative Binomial Regression with Template Model Builder

Version 0.1.0
Author and maintainer Zhenyu Wang <wonstran@hotmail.com>
Depends R (>= 4.1)
Imports stats, TMB, numDeriv
License MIT

Contents

1. [`rpbnb.tmb-package`](#)
2. [`fit\_rpbnb\_tmb`](#)
3. [`simulate\_rpbnb\_tmb`](#)
4. [`rpbnb\_tmb\_control`](#)
5. [`copula`](#)
6. [`rpbnb\_tmb\_marginal\_effects`](#)
7. [`rpbnb\_tmb\_elasticities`](#)
8. [`rpbnb\_tmb\_dependence\_profile`](#)
9. [`Methods for rpbnb\_tmb\_fit`](#)
10. [`Model, dependence, and random-coefficient notes`](#)
11. [`References and citation`](#)
12. [`License`](#)

Random-Parameter Bivariate Negative Binomial Regression

Description

`rpbnb.tmb` estimates bivariate count-regression models with negative-binomial margins, equation-specific covariates, optional random coefficients, and simulated likelihood integration based on randomized Halton draws. Estimation uses Template Model Builder (TMB) for automatic differentiation and `nlmminb()` for optimization.

Details

The two response equations may use different formulas. Dependence can be specified as Famoye/Sarmanov, independence, or a discrete copula. The fitting function returns an `rpbnb_tmb_fit` object with estimated coefficients, covariance information, TMB diagnostics, and the requested and realized thread counts.

Examples

```
library(rpbnb.tmb)

sim <- simulate_rpbnb_tmb(n = 20,
  beta1 = c("(Intercept)" = 0.2, x1 = 0.4),
  beta2 = c("(Intercept)" = 0.1, x1 = -0.3),
  dispersion = c(m1 = 0.4, m2 = 0.5), seed = 1)
ctrl <- rpbnb_tmb_control(n_cores = 1L)

# Not run during manual build:
# fit <- fit_rpbnb_tmb(y1 ~ x1, y2 ~ x1, data = sim$data,
#   draws = 20L, control = ctrl)
# summary(fit)
```

Fit a bivariate random-parameter negative binomial model

Description

Fits a maximum simulated likelihood model with two negative-binomial count margins. Random-coefficient distributions are integrated over Halton draws, while TMB supplies objective and derivative calculations.

Usage

```
fit_rpbnb_tmb(formula_1, formula_2, data,
              random_1 = NULL, random_2 = NULL,
              draws = 400L, seed = 1234L, start = NULL,
              dependence = "famoye",
              control = rpbnb_tmb_control(),
              inference = c("full", "diag", "none"),
              keep = c("postfit", "compact", "full"),
              poisson_1 = FALSE, poisson_2 = FALSE)
```

Arguments

formula_1, formula_2

Model formulas for the first and second count outcomes.

data

A data frame containing the outcomes and all variables named by both formulas. Incomplete rows for the modeled variables are omitted.

random_1, random_2

Random-coefficient specifications for each equation. Use `NULL` for fixed coefficients, an unnamed character vector for Normal random coefficients, or a named list of distribution specifications. Supported distributions are "normal", "lognormal", "uniform", and "triangular".

draws

Number of randomized Halton simulation draws. The default is 400L.

seed

Integer seed used while generating the Halton randomization.

start

Optional named or positional starting vector. Names must match the model's parameter names when supplied.

dependence

"famoye", "independence", or an object returned by `copula()`.

control

A control object from `rpbnb_tmb_control()`.

inference

"full" for a full covariance matrix, "diag" for standard errors only (`vcov()` then returns NA for cross-terms), or "none" to skip Hessian calculations entirely.

keep

Retained fit state: "postfit" keeps data needed for marginal effects, "compact" keeps estimates only, and "full" also retains the TMB objective and responses. Low-level diagnostics that access `fit$obj` -- including the profile method of `rpbnb_tmb_dependence_profile` -- require "full".

poisson_1, poisson_2

Logical flags fixing the selected margin at its Poisson-limit dispersion.

Value

An object of class `rpbnb_tmb_fit`. Important components include `coef`, `vcov`, `se`, `logLik`, `nobs`, `npar`, `m1`, `m2`, `optimizer`, `sdreport`, and `parallel`. The `parallel` component records the requested and realized TMB thread counts.

For Famoye dependence, `lambda_bounds` gives the admissible `c(lower =, upper =)` interval, computed once at the starting values and held fixed for the whole fit. `boundary_report` names any reported quantity pinned against an implementation bound -- the frozen Famoye interval, the Frank overflow guard, a saturated Gaussian link, or a Clayton `exp()` clamp -- or whose delta-method derivative has collapsed numerically, with `boundary_sides` recording which end ("lower", "upper", or "degenerate") was reached. Those estimates are set by the implementation rather than identified by the data, so their standard errors are reported as NA and a warning is raised; see `rpbnb_tmb_dependence_profile` for a valid interval instead.

Details

For copula dependence, Poisson-limit margins are not supported. The fitting routine checks the random-coefficient names against their equation's model matrix and validates the allowed dependence specification before constructing the TMB objective. Before building the TMB tape, it also checks the fit's weighted workload (`n * draws`, family-weighted) against `control$max_workload` and errors if the budget is exceeded; see `rpbnb_tmb_control` for the memory-guard details.

Examples

```
sim <- simulate_rpbnb_tmb(n = 20,
  beta1 = c("(Intercept)" = 0.2, x1 = 0.4),
  beta2 = c("(Intercept)" = 0.1, x1 = -0.3),
  dispersion = c(m1 = 0.4, m2 = 0.5), seed = 1)

# Not run during manual build:
# fit <- fit_rpbnb_tmb(y1 ~ x1, y2 ~ x1, data = sim$data,
#   random_1 = "x1", draws = 20L,
#   dependence = "famoye",
#   control = rpbnb_tmb_control(n_cores = 1L))
```

See Also

`rpbnb_tmb_control`, `copula`, `methods` for fitted objects, `rpbnb_tmb_dependence_profile`.

Simulate bivariate negative-binomial data

Description

Generates bivariate count data from a random-parameter negative-binomial process with equation-specific coefficients and a selected dependence mechanism.

Usage

```
simulate_rpbnb_tmb(n, beta1, beta2,  
                  random_1 = NULL, random_2 = NULL,  
                  dispersion = c(m1 = 0.5, m2 = 0.5),  
                  dependence = "famoye", lambda = 0,  
                  covariates = NULL, seed = NULL)
```

Arguments

n

Number of observations to generate.

beta1, beta2

Named coefficient vectors for the two equations. Each must include "(Intercept)".

random_1, random_2

Random-coefficient specifications in the same format accepted by `fit_rpbnb_tmb()`. Simulation requires a finite positive scale for each requested random coefficient.

dispersion

Named vector with positive `m1` and `m2` values.

dependence

"famoye", "independence", or a `copula()` object.

lambda

Famoye dependence parameter. The default zero produces independent draws.

covariates

Optional data frame. When omitted, required covariates are generated from standard Normal distributions.

seed

Optional integer seed.

Value

A list containing `data`, `truth`, realized coefficient matrices, fitted conditional means, and simulation settings.

Details

For the Famoye model, a nonzero `lambda` must lie inside the model's admissible bounds. Copula simulation uses the selected discrete CDF rectangle construction.

Examples

```
sim <- simulate_rpbnb_tmb(n = 20,  
  beta1 = c("(Intercept)" = 0.2, x1 = 0.4),  
  beta2 = c("(Intercept)" = 0.1, x1 = -0.3),  
  dispersion = c(m1 = 0.4, m2 = 0.5),  
  dependence = "independence", seed = 42)  
head(sim$data)
```

See Also

[fit_rpbnb_tmb](#) and [copula](#).

Control parameters for rpbnb.tmb estimation

Description

Creates a validated control object for optimization, TMB verbosity, OpenMP thread requests, Halton-draw burn-in, and a memory-usage guard on TMB tape construction.

Usage

```
rpbnb_tmb_control(iterlim = 500L, reltol = 1e-8,  
                  print_level = 0L, n_cores = 1L,  
                  max_threads = 4L,  
                  max_workload = .calibration_default_workload(),  
                  parallel_tape = FALSE,  
                  halton_burn = 300L)
```

Arguments

iterlim

Maximum optimizer iterations.

reltol

Relative convergence tolerance passed to `nlmminb()`.

print_level

Nonnegative verbosity level used by the fitting process.

n_cores

One whole-number TMB/OpenMP thread request. The installed TMB DLL can realize fewer threads; inspect `fit$parallel` after fitting.

max_threads

Maximum OpenMP threads permitted for one fit; `n_cores` requests above this are capped. Raise it explicitly to opt into a larger memory footprint.

max_workload

Maximum weighted observation-draw evaluations ($n * \text{draws}$, family-weighted) permitted before TMB tape construction; `Inf` disables the guard. The default targets a peak working set of about 8 GiB for one fit, calibrated by `inst/benchmark_memory.R`. Family weights relative to Famoye (1): independence 0.7, Frank 2.9, Gaussian 1.1, Clayton 1. With `parallel_tape = TRUE` the guard multiplies the workload by the realized thread count, since tapes are then built concurrently.

parallel_tape

Construct per-thread TMB tapes concurrently. The default `FALSE` builds them sequentially to reduce peak memory; objective and gradient evaluation remain parallel regardless.

halton_burn

Number of leading Halton points discarded before simulation draws are used.

Value

An object of class `rpbnb_tmb_control`.

Examples

```
ctrl <- rpbnb_tmb_control(iterlim = 300L, n_cores = 1L)
ctrl$n_cores
```

See Also

[fit_rpbmb_tmb](#).

Specify a dependence copula

Description

Constructs a copula specification for use as the dependence argument of `fit_rpbmb_tmb()` or `simulate_rpbmb_tmb()`.

Usage

```
copula(family, par = NULL)
```

Arguments

family

One of "frank", "normal", or "kimeldorf".

par

Optional fixed family parameter. For the Normal family, $|\rho| < 1$; for the Kimeldorf/Clayton family, $\theta > 0$; and for Frank, the parameter must be finite.

Value

An object of class `rpbnb_copula` containing the selected family and optional parameter.

Details

With an unspecified parameter, the dependence parameter is estimated by the fitting routine. Use "independence" rather than a copula object when an independent model is required.

Examples

```
copula("frank")
copula("normal", par = 0.3)
copula("kimeldorf", par = 1.2)
```

See Also

[fit_rpbmb_tmb](#) and [model notes](#).

Marginal effects for a fitted `rpbnb.tmb` model

Description

Computes average marginal effects (AME) or marginal effects at the mean (MEM). Continuous variables use derivatives of the integrated conditional mean; binary variables use discrete differences.

Usage

```
rpbnb_tmb_marginal_effects(fit, which = c("y1", "y2", "both"),
                           type = c("AME", "MEM"), vars = NULL,
                           include_intercept = FALSE, digits = 4L, ...)
```

Arguments

fit

An `rpbnb_tmb_fit` object.

which

Margin "y1", "y2", or "both".

type

"AME" or "MEM".

vars

Optional variable names to retain.

include_intercept

Whether to include the intercept row.

digits

Printed decimal places.

Value

A data frame for one margin or a named list of two data frames when `which = "both"`.

Details

When a coefficient is random, the computation uses the model's stored simulation draws to integrate the conditional mean and derivative.

Examples

```
# Not run during manual build:
# rpbnb_tmb_marginal_effects(fit, which = "both", type = "AME")
```

See Also

[rpbnb_tmb_elasticities](#).

Elasticities for a fitted `rpbnb.tmb` model

Description

Computes continuous elasticities and binary semi-elasticities from the same Monte Carlo integrated means used for marginal effects.

Usage

```
rpbnb_tmb_elasticities(fit, which = c("y1", "y2", "both"),
                      type = c("AME", "MEM"), vars = NULL,
                      include_intercept = FALSE, digits = 4L, ...)
```

Arguments

The arguments have the same meaning as for `rpbnb_tmb_marginal_effects`.

Value

A data frame for one margin or a named list of two data frames for both margins.

Details

For continuous variables, the reported quantity is $x / E[Y] * dE[Y]/dx$. For binary variables, the reported semi-elasticity is $E[Y \mid x = 1] / E[Y \mid x = 0] - 1$.

Examples

```
# Not run during manual build:
# rpbnb_tmb_elasticities(fit, which = "y1", type = "AME")
```

See Also

`rpbnb_tmb_marginal_effects`.

Confidence interval for a fitted dependence parameter

Description

Returns a profile-likelihood interval for the dependence parameter of a fitted model, computed on the unconstrained working scale and mapped through the family's monotone link. This is the tool to reach for when `summary()` reports NA for the dependence standard error.

Usage

```
rpbnb_tmb_dependence_profile(fit, level = 0.95,  
                             method = c("profile", "wald"), ...)
```

Arguments

fit

An `rpbnb_tmb_fit` object.

level

Coverage level; one number strictly between 0 and 1. Default 0.95.

method

"profile" (default) for a profile-likelihood interval, or "wald" for a Wald interval on the working scale mapped through the same link. "profile" requires the TMB objective, retained only under `keep = "full"`; when unavailable this degrades to "wald" with a warning naming the actual cause, rather than failing.

...

Passed to TMB: `tmbprofile()`, e.g. `ytol` or `parm.range`. `lincomb` and `slice` are rejected: the first would silently profile a different quantity than `z_dep`, and the second returns a likelihood slice rather than a profile.

Value

A data frame with one row per reported dependence quantity and columns `parameter`, `estimate`, `lower`, `upper`, `level`, and `method` -- the method actually used, which may differ from the one requested. The raw profile, when one was computed, is attached as `attr(, "profile")` so it can be plotted.

Details

At a boundary the delta-method derivative collapses, so $SE = |d\theta/dz| * SE(z)$ is a zero-times-infinity product and no symmetric standard error exists; a profile interval needs no derivative. For Famoye this is the usual situation: `lambda_bounds` is frozen at the starting values, so a pinned estimate's interval is still bounded by that same frozen box, not an escape from it -- widen it first by refitting from better starting values.

Examples

```
# Not run during manual build:  
# rpbnb_tmb_dependence_profile(fit, level = 0.95)
```

See Also

[fit_rpbnb_tmb.](#)

Methods for `rpbnb_tmb_fit` objects

Description

Standard methods expose fitted-model quantities and concise reporting for objects returned by `fit_rpbnb_tmb()`.

Usage

```
print(x, ...)
summary(object, digits = 4L, ...)
coef(object, ...)
vcov(object, ...)
logLik(object, ...)
AIC(object, ...)
BIC(object, ...)
predict(object, newdata = NULL,
        type = c("response", "link"),
        which = c("both", "y1", "y2"), ...)
```

Details

- `print()` reports the likelihood, observation and parameter counts, realized TMB threads, dependence specification, and rounded coefficients.
- `summary()` prints equation coefficients, random-coefficient scale summaries, dispersions, and dependence information when available from `sdreport`. A dependence or dispersion estimate pinned against an implementation bound (see `boundary_report` under `fit_rpbnb_tmb`) prints its standard error as NA rather than a misleading number; use `rpbnb_tmb_dependence_profile` for a valid interval instead.
- `coef()` returns the named parameter vector and `vcov()` returns the fitted covariance matrix.
- `logLik()` attaches degrees of freedom and observation count; `AIC()` and `BIC()` use the stored log likelihood and parameter count.
- `predict()` returns fitted response means (or, with `type = "link"`, their log) for the retained fitting design or a supplied `newdata`, integrating over retained simulation draws when random coefficients are present. The result is a two-column numeric matrix for `which = "both"`, otherwise a numeric vector. Compact fits (`keep = "compact"`) require `newdata`, since the fitting design is not retained.

Examples

```
# Not run during manual build:
# coef(fit)
# vcov(fit)
# logLik(fit)
# AIC(fit)
# BIC(fit)
# predict(fit)
```

See Also

`fit_rpbnb_tmb`, `rpbnb_tmb_dependence_profile`.

Model, dependence, and random-coefficient notes

Negative-binomial margins

Each equation has its own mean specification and NB2 dispersion parameter. Set `poisson_1` or `poisson_2` in `fit_rpbnb_tmb()` only for a requested Poisson-limit margin.

Dependence choices

"famoye" estimates a Famoye/Sarmanov dependence parameter subject to admissible bounds.

"independence" omits the estimated dependence parameter. Copula specifications use Frank, Gaussian, or Kimeldorf/Clayton discrete-copula dependence. A dependence estimate pinned at an implementation bound `--lam` at a Famoye bound, `|theta|` at the Frank overflow guard, `|rho| = 1` for Gaussian, or the Clayton `exp()` clamp `--reports` its standard error as NA because the estimate is set by the implementation rather than identified by the data; see `rpbnb_tmb_dependence_profile` for a profile-likelihood or Wald interval instead.

Random coefficients

Random coefficients are equation-specific and their variable names must occur in the corresponding formula. An unnamed character vector requests Normal distributions. A named list can select Normal, lognormal, uniform, or triangular distributions; a lognormal specification may include `sign = -1` or `sign = 1`.

Threading note. `n_cores` requests an OpenMP thread count for the loaded TMB DLL. The realized count can be lower on a given build or platform. Inspect `fit$parallel$requested` and `fit$parallel$realized` after fitting.

Memory guard. Estimation memory scales with `n * draws` (family-weighted). `rpbnb_tmb_control()`'s `max_workload` rejects fits above a calibrated peak-memory budget before the TMB tape is built, and the default `parallel_tape = FALSE` keeps that peak from scaling with thread count. Raise `max_workload` deliberately against the memory you actually have; see `rpbnb_tmb_control`.

References and citation

The implementation uses Template Model Builder for automatic differentiation and Laplace-style reporting infrastructure. Cite the package as software using its package name, version, author, and repository URL:

Wang, Z. (2026). `rpbnb.tmb`: Random-Parameter Bivariate Negative Binomial Regression with Template Model Builder. R package version 0.1.0.

For TMB methodology, see Kristensen, K., Nielsen, A., Berg, C. W., Skaug, H., and Bell, B. M. (2016). TMB: Automatic Differentiation and Laplace Approximation. *Journal of Statistical Software*, 70(5), 1-21.

License

MIT License. Copyright (c) 2026 Zhenyu Wang. Permission is granted, free of charge, to use, copy, modify, merge, publish, distribute, sublicense, and sell copies of the software, subject to inclusion of the copyright and permission notice. The software is provided without warranty.